Doomscroll

Use Case Model — Lab 2 (refined)

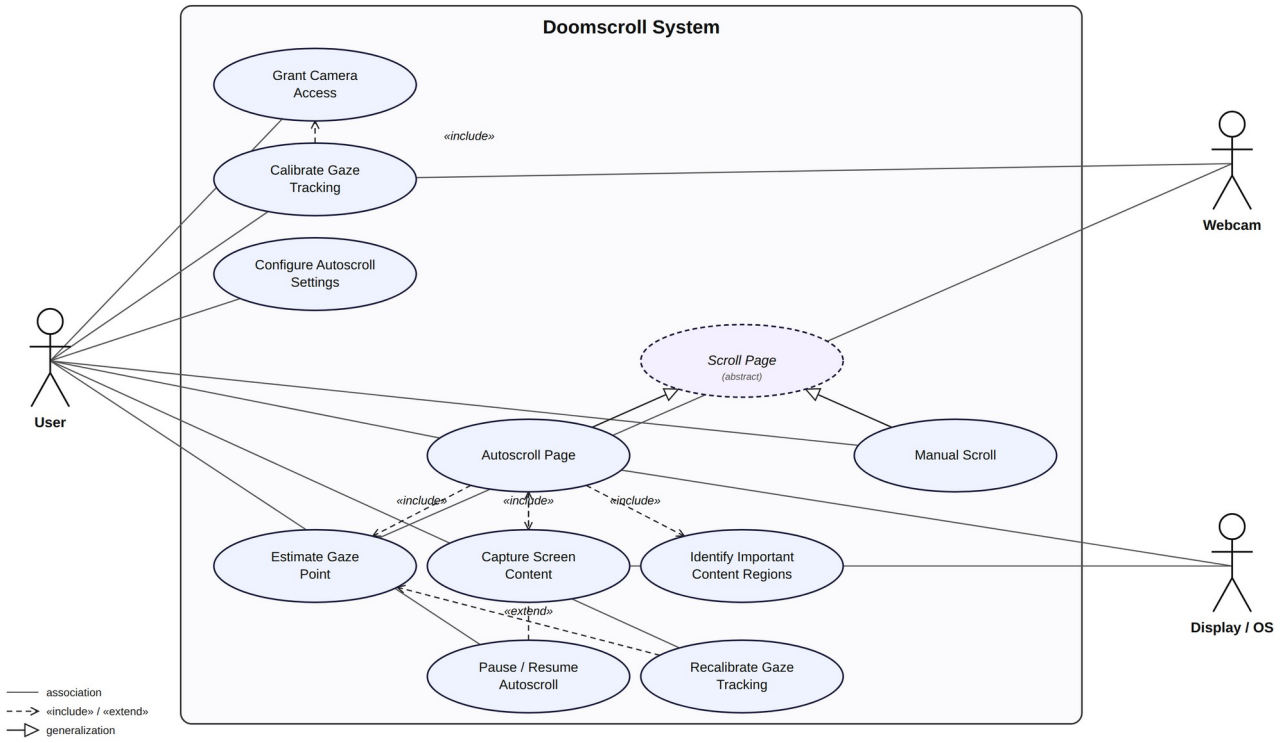
Actors

User (primary): the person reading on-screen content whose head and gaze are tracked, and who configures and controls the application.

Webcam (secondary): the device's camera hardware, supplying the images the system analyses.

Display / OS (secondary): the operating system's screen buffer and scroll functions — the source of Screen Content and the target of scrolling.

Use Case Diagram



Use Case IDs below are grouped hierarchically by the area of functionality they belong to: 1.x Camera Access and Calibration, 2.x Gaze Tracking, 3.x Screen Content Analysis, 4.x Scrolling, and 5.x User Control and Configuration. «include» and «extend» relationships from the diagram are represented in each use case's Includes field or Alternative Flows field respectively, following the Use Case Template.

Use Case Descriptions

Per Lab 2 Section 3.1, these use case descriptions have been reviewed for consistency with the conceptual model (class_diagram.jpg) and dialog map (dialog_map.jpg) developed for Lab 2; control-class names (e.g. GazeEstimationController, AutoscrollController, CalibrationController) trace directly back to the corresponding use cases below (Estimate Gaze Point, Autoscroll Page, Calibrate Gaze Tracking, etc.). No substantive changes to flows were needed at this stage — refinement continues in Lab 3.
Each use case below is documented using the Use Case Template.

Use Case 1.1 — Grant Camera Access

Use Case ID:	1.1		
Use Case Name:	Grant Camera Access		
Created By:	Project Team	Last Updated By:	Project Team
Date Created:	6 September 2026	Date Last Updated:	20 September 2026

Actor:	User
Description:	The User grants the application permission to access the device's webcam, which is required before any gaze-tracking function can operate.
Preconditions:	1. The application has been launched for the first time, or camera permission was previously withdrawn.
Postconditions:	1. The system knows whether it has camera permission, and Autoscroll functions are enabled or disabled accordingly.
Priority:	High
Frequency of Use:	Once per installation, or whenever camera permission has been withdrawn.
Flow of Events:	1. The system detects that it does not currently have permission to access the webcam. 2. The system displays a request explaining why camera access is needed. 3. The User responds to the operating system's permission dialog, either allowing or denying access. 4. The system receives the User's decision. 5. If access is allowed, the system enables Use Case 1.2 (Calibrate Gaze Tracking).
Alternative Flows:	1.1.AC.1: If the User denies camera access, the system disables all Autoscroll functions and displays a message explaining that camera access is required.
Exceptions:	None.
Includes:	None.
Special Requirements:	Requirement 3.1.1; Requirement 3.1.2.1; Requirement 3.1.2.2.
Assumptions:	The User's device has a working webcam and an operating system capable of presenting a camera-permission prompt.
Notes and Issues:	None.

Use Case 1.2 — Calibrate Gaze Tracking

Use Case ID:	1.2		
Use Case Name:	Calibrate Gaze Tracking		
Created By:	Project Team	Last Updated By:	Project Team
Date Created:	6 September 2026	Date Last Updated:	20 September 2026

Actor:	User
Description:	The User completes a guided Calibration sequence so the system can build a Calibration Profile that maps the User's head and eye movements to positions on their screen.

Preconditions:	1. Camera access has been granted (Use Case 1.1).
Postconditions:	1. A Calibration Profile exists for the User and is stored for future Sessions.
Priority:	High
Frequency of Use:	Once per user, and again whenever the User chooses to recalibrate.
Flow of Events:	1. The system displays a sequence of points across the screen, one at a time. 2. The User looks at each point as it is highlighted. 3. The system observes the User's Head Pose and Gaze Direction while the User looks at each point. 4. The system builds a Calibration Profile from the observations collected at all points. 5. The system stores the Calibration Profile for the User. 6. The system confirms that Calibration is complete and enables Autoscroll.
Alternative Flows:	1.2.AC.1: If the User chooses to skip Calibration, the system leaves Autoscroll disabled until Calibration is later completed.
Exceptions:	1.2.EX.1: If the system cannot reliably observe the User's Head Pose or Gaze Direction at a given point (for example, due to poor lighting), the system asks the User to look at that point again before continuing.
Includes:	Use Case 1.1 (Grant Camera Access).
Special Requirements:	Requirement 3.1.3; Requirement 3.1.4.
Assumptions:	The User is seated at a normal viewing distance from the screen during Calibration.
Notes and Issues:	None.

Use Case 1.3 — Recalibrate Gaze Tracking

Use Case ID:	1.3		
Use Case Name:	Recalibrate Gaze Tracking		
Created By:	Project Team	Last Updated By:	Project Team
Date Created:	6 September 2026	Date Last Updated:	20 September 2026

Actor:	User
Description:	The User repeats Calibration after the system detects that the Confidence Score of the User's Gaze Point has remained low for a sustained period, typically because lighting or the User's position relative to the screen has changed.
Preconditions:	1. The Confidence Score of the User's Gaze Point has remained below the defined threshold for the specified duration (see Use Case 2.1).
Postconditions:	1. Either a new Calibration Profile has been created for the User, or the User has explicitly chosen to continue without recalibrating.
Priority:	Medium
Frequency of Use:	As needed; expected to be infrequent for a User in a stable setup.
Flow of Events:	1. The system pauses Autoscroll and displays a prompt asking the User to recalibrate. 2. The User chooses to recalibrate now. 3. The system carries out Use Case 1.2 (Calibrate Gaze Tracking).

	4. The system resumes Autoscroll using the new Calibration Profile.
Alternative Flows:	1.3.AC.1: If the User dismisses the prompt instead of recalibrating, the system continues using the existing Calibration Profile and will prompt the User again later if low confidence continues.
Exceptions:	None.
Includes:	Use Case 1.2 (Calibrate Gaze Tracking).
Special Requirements:	Requirement 3.1.5.1; Requirement 3.1.5.2.
Assumptions:	None.
Notes and Issues:	None.

Use Case 2.1 — Estimate Gaze Point

Use Case ID:	2.1		
Use Case Name:	Estimate Gaze Point		
Created By:	Project Team	Last Updated By:	Project Team
Date Created:	6 September 2026	Date Last Updated:	20 September 2026

Actor:	User
Description:	The system continuously determines the point on the screen the User is currently looking at, so that this Gaze Point can be used to decide when to trigger Autoscroll.
Preconditions:	1. Camera access has been granted. 2. A Calibration Profile exists for the User.
Postconditions:	1. A current Gaze Point and an associated Confidence Score are available to Use Case 4.1 (Autoscroll Page).
Priority:	High
Frequency of Use:	Continuously while the application is active (multiple times per second).
Flow of Events:	1. The system observes the User's Head Pose. 2. The system observes the User's Gaze Direction. 3. The system combines the Head Pose, Gaze Direction, and the User's Calibration Profile to compute a Gaze Point. 4. The system computes a Confidence Score for the Gaze Point. 5. If the Confidence Score meets the minimum threshold, the system makes the Gaze Point available to Use Case 4.1 (Autoscroll Page).
Alternative Flows:	2.1.AC.1: If the Confidence Score remains below the minimum threshold for a sustained period, see Use Case 1.3 (Recalibrate Gaze Tracking).
Exceptions:	2.1.EX.1: If the User's face cannot be detected in the webcam image, the system produces no Gaze Point for that instant and Autoscroll remains paused until the face is detected again.
Includes:	None.
Special Requirements:	Requirement 3.2.1; Requirement 3.2.2; Requirement 3.2.3; Requirement 3.2.4; Requirement 3.2.5; Requirement 4.9.1.

Assumptions:	None.
Notes and Issues:	None.

Use Case 3.1 — Capture Screen Content

Use Case ID:	3.1		
Use Case Name:	Capture Screen Content		
Created By:	Project Team	Last Updated By:	Project Team
Date Created:	6 September 2026	Date Last Updated:	20 September 2026

Actor:	User
Description:	The system periodically examines the Screen Content of the active window so that it can be analysed for Important Content and Unimportant Content.
Preconditions:	1. The application has permission to read the contents of the screen or active window.
Postconditions:	1. A current view of the Screen Content is available to Use Case 3.2 (Identify Important Content Regions).
Priority:	High
Frequency of Use:	Continuously while the application is active, and immediately after each detected content change.
Flow of Events:	1. The system reads the current Screen Content of the active window. 2. The system compares the current Screen Content with the previously examined Screen Content. 3. If the content has changed materially, the system marks the Screen Content as needing re-classification and makes it available to Use Case 3.2 (Identify Important Content Regions).
Alternative Flows:	None.
Exceptions:	3.1.EX.1: If the system does not have permission to read the screen or active window, the system disables content-aware Autoscroll and notifies the User.
Includes:	None.
Special Requirements:	Requirement 3.3.1; Requirement 3.3.5.
Assumptions:	None.
Notes and Issues:	None.

Use Case 3.2 — Identify Important Content Regions

Use Case ID:	3.2		
Use Case Name:	Identify Important Content Regions		
Created By:	Project Team	Last Updated By:	Project Team
Date Created:	6 September 2026	Date Last Updated:	20 September 2026

Actor:	User
Description:	The system classifies the current Screen Content into Important Content and Unimportant Content, and determines the Content Boundary of the Important Content, so that Autoscroll can be triggered at the right point.
Preconditions:	1. A current view of the Screen Content is available (Use Case 3.1).
Postconditions:	1. The current Content Boundary of the Important Content is available to Use Case 4.1 (Autoscroll Page).
Priority:	High
Frequency of Use:	Whenever the Screen Content changes materially.
Flow of Events:	1. The system examines the current Screen Content. 2. The system identifies which parts of the Screen Content are Important Content. 3. The system identifies which parts of the Screen Content are Unimportant Content. 4. The system determines the Content Boundary of the Important Content. 5. The system makes the Content Boundary available to Use Case 4.1 (Autoscroll Page).
Alternative Flows:	None.
Exceptions:	None.
Includes:	None.
Special Requirements:	Requirement 3.3.2; Requirement 3.3.3; Requirement 3.3.4.
Assumptions:	None.
Notes and Issues:	None.

Use Case 4.0 — Scroll Page (abstract)

Use Case ID:	4.0		
Use Case Name:	Scroll Page (abstract)		
Created By:	Project Team	Last Updated By:	Project Team
Date Created:	6 September 2026	Date Last Updated:	20 September 2026

Actor:	User
Description:	An abstract use case representing the shared goal of moving the visible Screen Content, achieved either automatically (Use Case 4.1, Autoscroll Page) or manually (Use Case 4.2, Manual Scroll). This use case has no Flow of Events of its own; each of its two concrete forms supplies its own trigger and flow.
Preconditions:	1. Not applicable — see Use Case 4.1 and Use Case 4.2.
Postconditions:	1. Not applicable — see Use Case 4.1 and Use Case 4.2.
Priority:	N/A
Frequency of Use:	N/A
Flow of Events:	1. Not applicable. This is an abstract use case; see Use Case 4.1 (Autoscroll Page) and Use Case 4.2 (Manual Scroll) for concrete flows of events.
Alternative Flows:	None.

Exceptions:	None.
Includes:	None.
Special Requirements:	None.
Assumptions:	None.
Notes and Issues:	Use Case 4.1 and Use Case 4.2 are both specialisations of this use case.

Use Case 4.1 — Autoscroll Page

Use Case ID:	4.1		
Use Case Name:	Autoscroll Page		
Created By:	Project Team	Last Updated By:	Project Team
Date Created:	6 September 2026	Date Last Updated:	20 September 2026

Actor:	User
Description:	The system scrolls the Screen Content automatically, in the Scroll Direction corresponding to the Content Boundary the User's Gaze Point has reached, so the User can keep reading without performing a Manual Scroll. This use case is a specialisation of Use Case 4.0 (Scroll Page).
Preconditions:	1. Calibration is complete. 2. Autoscroll is not paused.
Postconditions:	1. The Screen Content has scrolled in the appropriate Scroll Direction, or remains unchanged if the Gaze Point was not near a Content Boundary.
Priority:	High
Frequency of Use:	Continuously while the application is active and Autoscroll is not paused.
Flow of Events:	1. The system obtains the current Gaze Point and its Confidence Score from Use Case 2.1 (Estimate Gaze Point). 2. The system obtains the current Content Boundary from Use Case 3.2 (Identify Important Content Regions). 3. The system checks whether the Gaze Point is within the configured distance of a Content Boundary. 4. If so, the system scrolls the Screen Content in the corresponding Scroll Direction, at the speed configured in the Autoscroll Settings. 5. The system continues monitoring the Gaze Point. 6. When the Gaze Point moves back into the Important Content, the system stops scrolling.
Alternative Flows:	4.1.AC.1: If the User pauses or resumes Autoscroll at any point during this flow, see Use Case 5.2 (Pause / Resume Autoscroll).
Exceptions:	None.
Includes:	Use Case 2.1 (Estimate Gaze Point); Use Case 3.1 (Capture Screen Content); Use Case 3.2 (Identify Important Content Regions).
Special Requirements:	Requirement 3.4.1; Requirement 3.4.2; Requirement 3.4.3; Requirement 3.4.4.
Assumptions:	None.

Notes and Issues:	None.

Use Case 4.2 — Manual Scroll

Use Case ID:	4.2		
Use Case Name:	Manual Scroll		
Created By:	Project Team	Last Updated By:	Project Team
Date Created:	6 September 2026	Date Last Updated:	20 September 2026

Actor:	User
Description:	The User scrolls the Screen Content directly, using the mouse, trackpad, or keyboard. This use case is a specialisation of Use Case 4.0 (Scroll Page), and takes precedence over Autoscroll while it occurs.
Preconditions:	1. The application is running.
Postconditions:	1. The Screen Content reflects the User's Manual Scroll, and Autoscroll resumes normal operation shortly afterward.
Priority:	Medium
Frequency of Use:	As often as the User chooses to scroll manually.
Flow of Events:	1. The User scrolls using the mouse, trackpad, or keyboard. 2. The system scrolls the Screen Content accordingly. 3. The system detects that a Manual Scroll has occurred. 4. The system suppresses Autoscroll for a short period, so the two do not conflict. 5. After that period, the system resumes normal Autoscroll operation.
Alternative Flows:	None.
Exceptions:	None.
Includes:	None.
Special Requirements:	Requirement 3.4.5; Requirement 3.4.6.
Assumptions:	None.
Notes and Issues:	None.

Use Case 5.1 — Configure Autoscroll Settings

Use Case ID:	5.1		
Use Case Name:	Configure Autoscroll Settings		
Created By:	Project Team	Last Updated By:	Project Team
Date Created:	6 September 2026	Date Last Updated:	20 September 2026

Actor:	User
Description:	The User views and changes the Autoscroll Settings that control Autoscroll behaviour, such as which Scroll Directions are enabled, sensitivity, and speed.

Preconditions:	1. The application is running.
Postconditions:	1. The updated Autoscroll Settings are stored and take effect immediately.
Priority:	Medium
Frequency of Use:	Occasionally, typically during initial setup and infrequently afterward.
Flow of Events:	1. The User opens the settings interface. 2. The User enables or disables one or more Scroll Directions. 3. The User adjusts the sensitivity setting. 4. The User adjusts the Autoscroll speed setting. 5. The User optionally sets a keyboard shortcut for pausing and resuming Autoscroll. 6. The system validates and stores the updated Autoscroll Settings.
Alternative Flows:	None.
Exceptions:	5.1.EX.1: If the User enters an invalid value (for example, a negative speed), the system rejects the value and asks the User to enter a valid one.
Includes:	None.
Special Requirements:	Requirement 3.5.1.1; Requirement 3.5.1.2; Requirement 3.5.1.3; Requirement 3.5.1.4; Requirement 3.5.2; Requirement 3.5.3; Requirement 4.10.1.
Assumptions:	None.
Notes and Issues:	None.

Use Case 5.2 — Pause / Resume Autoscroll

Use Case ID:	5.2		
Use Case Name:	Pause / Resume Autoscroll		
Created By:	Project Team	Last Updated By:	Project Team
Date Created:	6 September 2026	Date Last Updated:	20 September 2026

Actor:	User
Description:	The User pauses or resumes Autoscroll on demand, without closing the application. This use case extends Use Case 4.1 (Autoscroll Page).
Preconditions:	1. The application is running.
Postconditions:	1. Autoscroll is in the state (paused or active) requested by the User, and the on-screen indicator reflects that state.
Priority:	Medium
Frequency of Use:	As often as the User chooses to pause or resume.
Flow of Events:	1. The User presses the configured keyboard shortcut, or selects the on-screen control. 2. The system toggles Autoscroll between active and paused. 3. The system updates the on-screen indicator to reflect the new state. 4. If Autoscroll is resumed, the system re-enables Use Case 4.1 (Autoscroll Page).
Alternative Flows:	None.

Exceptions:	None.
Includes:	None.
Special Requirements:	Requirement 3.5.4.1; Requirement 3.5.4.2; Requirement 3.5.5.
Assumptions:	None.
Notes and Issues:	None.